

Despliegue

Censo Municipal de Animales · Navalcarnero

Despliegue

Dos modos soportados: con Docker (recomendado) y local sin Docker.

A. Despliegue con Docker

Requisitos

- Docker 24+ y Docker Compose v2.
- ~500 MB libres en disco.

Pasos

```
git clone <repo>
cd censo_animales
docker compose up -d --build
```

Esto arranca tres servicios:

Servicio	Imagen	Puertos	Descripción
nginx	nginx:alpine	80 / 8181	Web pública / panel admin
api	api/Dockerfile	(interno) 5000	Flask
mariadb	mariadb:10.11	(interno) 3306	Base de datos

URLs por defecto:

- `http://<IP-servidor>` — empleados y policía.
- `http://localhost:8181` — admin (sólo desde la propia máquina).

Variables de entorno (sección `api` en `docker-compose.yml`)

Variable	Por defecto	Descripción
DB_HOST	mariadb	Host de BD
DB_PORT	3306	Puerto
DB_USER	root	Usuario
DB_PASSWORD	123	Contraseña
DB_NAME	censo_animales	Nombre BD
LOG_DIR	logs	Directorio de logs
AUTH_FILE	auth.json	Fichero de credenciales/tokens
BACKUP_DIR	db/backups	Carpeta de backups
BACKUP_RETENTION_DAYS	30	Retención en días
FOTOS_ANIMALES_DIR	web/fotos_animales	Carpeta de fotos de animales

Variable	Por defecto	Descripción
FOTOS_INCIDENCIAS_DIR	web/fotos_incidencias	Carpeta de fotos de incidencias

Operaciones habituales

```
docker compose ps                # estado
docker compose logs -f api       # logs en vivo
docker compose down              # parar
docker compose down -v           # parar y borrar volúmenes (!destrutivo!)
docker compose exec mariadb mysql -uroot -p123 censo_animales # consola SQL
```

Aplicar una migración nueva

```
docker compose exec -T mariadb mysql -uroot -p123 censo_animales \
  < db/migrations/13_incidencias_foto.sql
docker compose restart api
```

B. Despliegue local sin Docker

Requisitos

- Python 3.10+
- MariaDB o MySQL (cliente y servidor) en 127.0.0.1:3306.
- Pip y venv.

Pasos

```
cd censo_animales
python -m venv .venv
. .venv/Scripts/activate      # Windows
# . .venv/bin/activate        # Linux/macOS
pip install -r api/requirements.txt
```

1) Crear BD y cargar dump inicial

```
mysql -uroot -p123 -e "CREATE DATABASE IF NOT EXISTS censo_animales DEFAULT CHARACTER SET utf8mb4;"
mysql -uroot -p123 censo_animales < db/censo_animales.sql
```

2) Aplicar migraciones en orden

```
for f in db/migrations/*.sql; do
  echo ">> $f"
  mysql -uroot -p123 censo_animales < "$f"
done
```

3) Arrancar

```
python run_local.py
```

Equivalente Windows PowerShell para aplicar migraciones:

```
Get-ChildItem db/migrations/*.sql | Sort-Object Name | ForEach-Object {
  Write-Host ">> $($_.Name)"
  Get-Content $_.FullName | mysql -uroot -p123 censo_animales
}
```

URLs:

- <http://localhost:8000> — empleados y policía.
- <http://localhost:8181> — admin.
- <http://127.0.0.1:5000/api/> — API directa (interno).

`run_local.py` reenvía `/api/*` al backend Flask, igual que nginx en el modo Docker.

Inicialización del administrador

La primera vez se accede a `http://localhost:8181` y se crea el usuario admin desde `/api/auth/setup`. Tras crear el admin, desde su panel se dan de alta empleados y agentes.

Producción real (sugerencias)

- Servir tras un proxy TLS (Caddy / Traefik / nginx) en HTTPS.
- Cambiar `DB_PASSWORD` y la contraseña inicial del admin.
- Restringir `localhost:8181` por VPN o firewall — el panel admin no debe

ser público.

- Programar copias del directorio `db/backups/` a almacenamiento externo.
- Habilitar rotación adicional para `logs/auditoria.jsonl` si el volumen lo

exige.

Estructura de directorios persistentes

<code>censo_animales/</code>	
■■■ <code>auth.json</code>	← credenciales/tokens (montar como secreto)
■■■ <code>logs/</code>	← <code>log.txt</code> + <code>auditoria.jsonl</code> (volumen)
■■■ <code>db/backups/</code>	← backups SQL (volumen)
■■■ <code>web/</code>	
■■■ <code>fotos_animales/</code>	← imágenes subidas (volumen)
■■■ <code>fotos_incidentes/</code>	← imágenes subidas (volumen)

Asegúrese de montar como volúmenes Docker estos paths para no perder datos en `docker compose down -v`.